

Lecture Note 6: ESP32 – MQTT Clients

CSS452 Internet of Things

Dr. Seksan Laitrakun

School of ICT, SIIT, Thammasat University

1. Introduction to MQTT
2. Installing a Necessary Library in Arduino IDE
3. Publishing/Subscribing via a Public MQTT Broker using Two ESP32
4. Publishing/Subscribing via a Public MQTT Broker using One ESP32

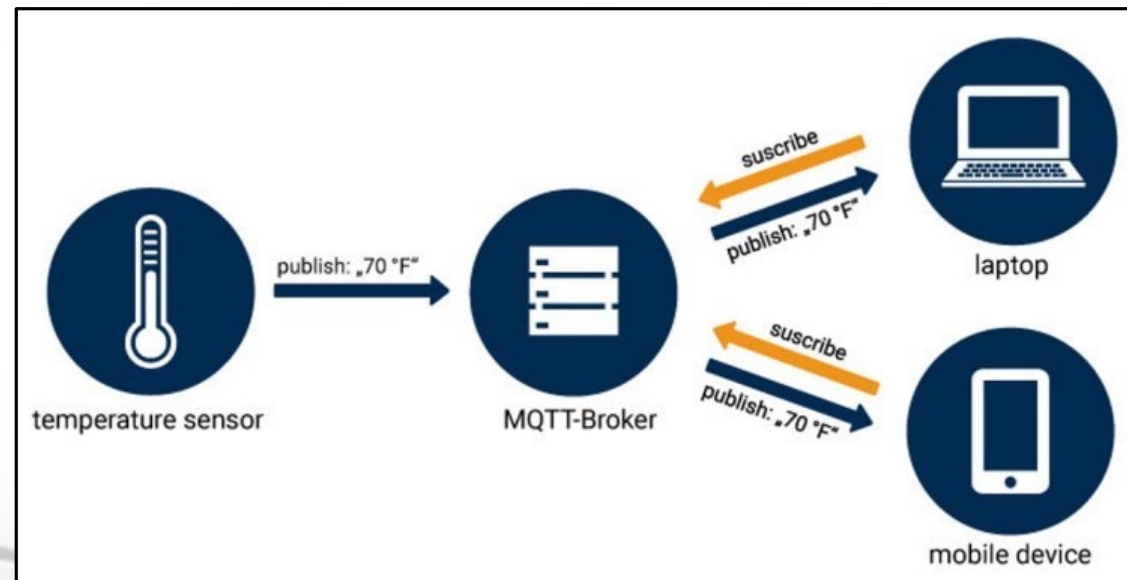
References

- Random Nerd Tutorials: <https://randomnerdtutorials.com/projects-esp32/>
- HiveMQ: <https://www.hivemq.com/developers/>

1. Introduction to MQTT

MQTT and Publish/Subscribe Method

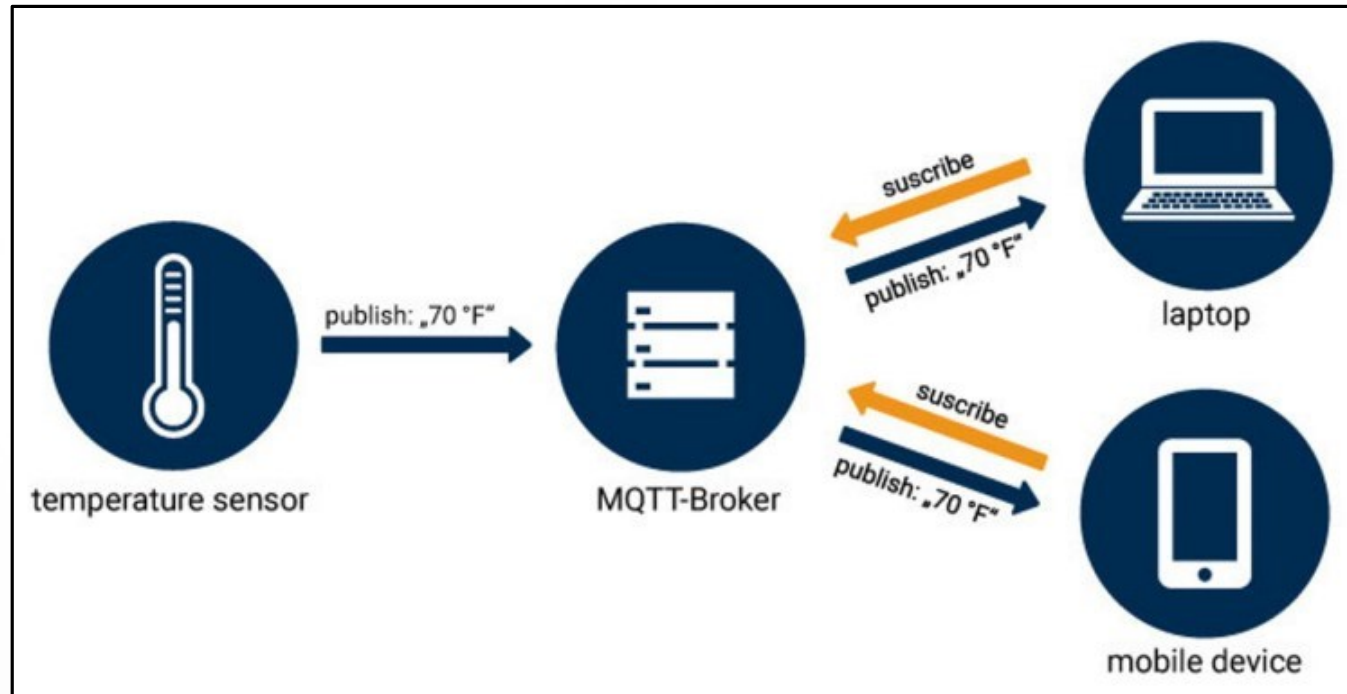
- **MQTT** (Message Queuing Telemetry Transport) is a communication protocol for the Internet of Things (IoT) that uses the **publish/subscribe method** for communication.
- MQTT is lightweight and is used in low bandwidth environments.
- It is a **popular** for IoT systems (which is a network of constrained devices).
- **Three** main components in MQTT:
 - ✓ Publisher/Producer (sender),
 - ✓ Broker (middleman)
 - ✓ Subscriber/Consumer (receiver).



MQTT and Publish/Subscribe Method

Publish/subscribe method:

- 1) The producer publishes a **message** (data) and the **topic** to the Broker.
- 2) The Broker receives the message (data) and the topic.
- 3) The Broker sends the message (data) to the appropriate consumer(s) that are subscribed to this **topic**.



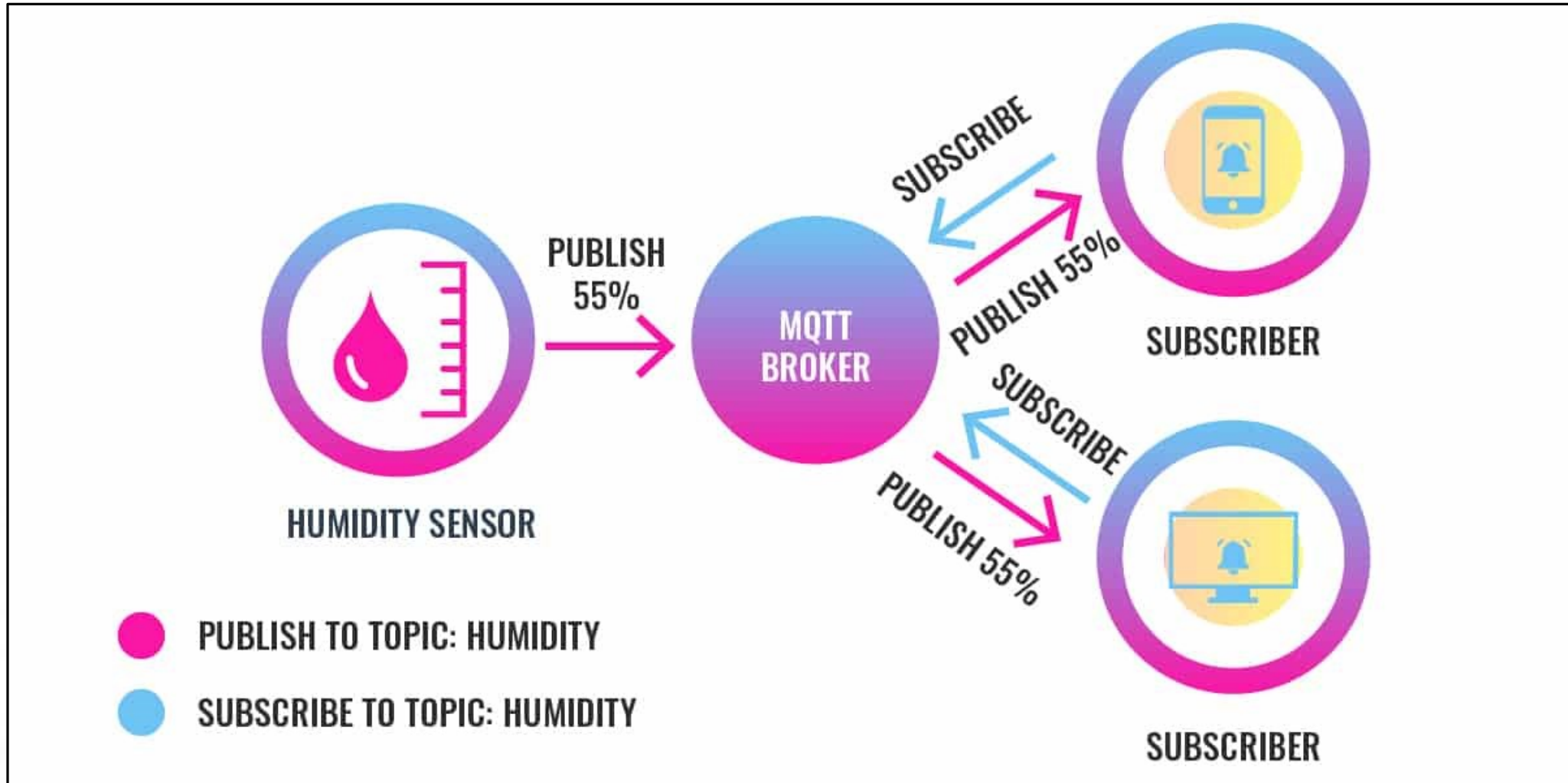
The **publish/subscribe** method (also known as pub/sub) provides an alternative to traditional client-server architecture.

- **HTTP protocol:** In the client-server model, a client communicates directly with an end point.
- **Advantages:** The pub/sub model decouples the client that sends a message (the publisher) from the client or clients that receive the messages (the subscribers).
 - **The publishers and subscribers never contact each other directly.** In fact, they are not even aware that the other exists. Unlike using the HTTP protocol (Lecture note 6), the publisher and subscriber do not need to know the IP addresses of each other.
 - **The connection between them is handled by a third component (the broker).** The job of the broker is to filter all incoming messages and distribute them correctly to subscribers.
 - The **topic** is the key such that the message can be delivered to the corresponding subscribers.
 - As a result, **the publisher and subscriber do not know each other.**

How does MQTT work?

- The publish/subscribe method solves the problem of detail, by introducing a new concept, i.e., **a topic**.
 - 1) Each publisher must declare itself to the MQTT broker and its published topics.
 - 2) Each subscriber must declare itself as a subscriber and its subscribed topics.
 - 3) The MQTT broker will distribute messages from publishers to subscribers according to the topic.
- In MQTT, an IoT device can be
 - a publisher,
 - a subscriber,
 - both publisher and subscriber.

Example



Topic Setup: A Systematic Way

Systematically, a topic can be defined as a **hierarchy** (or **level**), using a slash (/) as a separator.

- For example, a smart home (called “Home1”) consists 3 rooms: Room1, Room2, Room3. Each room consists to 2 sensor nodes: temperature node (called “temp”) and humidity node (called “humid”). Totally, we will have 6 sensor nodes to send these data. These sensor nodes might be set up as publishers and their topics could be

- Home1/Room1/temp
- Home1/Room1/humid
- Home1/Room2/temp
- Home1/Room2/humid
- Home1/Room3/temp
- Home1/Room3/humid

We have **3 topic levels** (we can create as many levels as we want).

- A device who want the data from the humidity data from room 2 will subscribe the topic named “Home1/Room2/humid”.

Topic Setup: A Systematic Way

By setting the topics in this way (hierarchy), we can use **two** wild cards when a devices would like to subscribe from many topics.

- 1) **The single-level wildcard**, indicated by **a plus sign (+)**, can be used to subscribe to various topics at once, but only one level deep.

Example:

Given the following published topics:

- Home1/Room1/temp
- Home1/Room1/humid
- Home1/Room2/temp
- Home1/Room2/humid
- Home1/Room3/temp
- Home1/Room3/humid

- a) Which topics will a subscriber receive if it subscribes to Home1/Room1/+?
- b) Which topics will a subscriber receive if it subscribes to Home1/+/temp?

- 2) **Multi-level wildcards**, indicated by **a number/hashtag character (#)**, can be used to subscribe to various topics at once – as the name suggests – multiple levels deep.

Example:

Given the following published topics:

- Home1/Room1/temp
- Home1/Room1/humid
- Home1/Room2/temp
- Home1/Room2/humid
- Home1/Room3/temp
- Home1/Room3/humid

- a) Which topics will a subscriber receive if it subscribes to Home1/Room1/#?
- b) Which topics will a subscriber receive if it subscribes to Home1/#?

- **What is QoS?** A setting that defines the **guarantee** of message delivery between the client and the broker.
- **Who decides?** The **publisher** sets the level for the message, and the **subscriber** sets the maximum level it is willing to receive.
- **The Rule:** The message is always delivered at the **lower** of the two QoS levels.

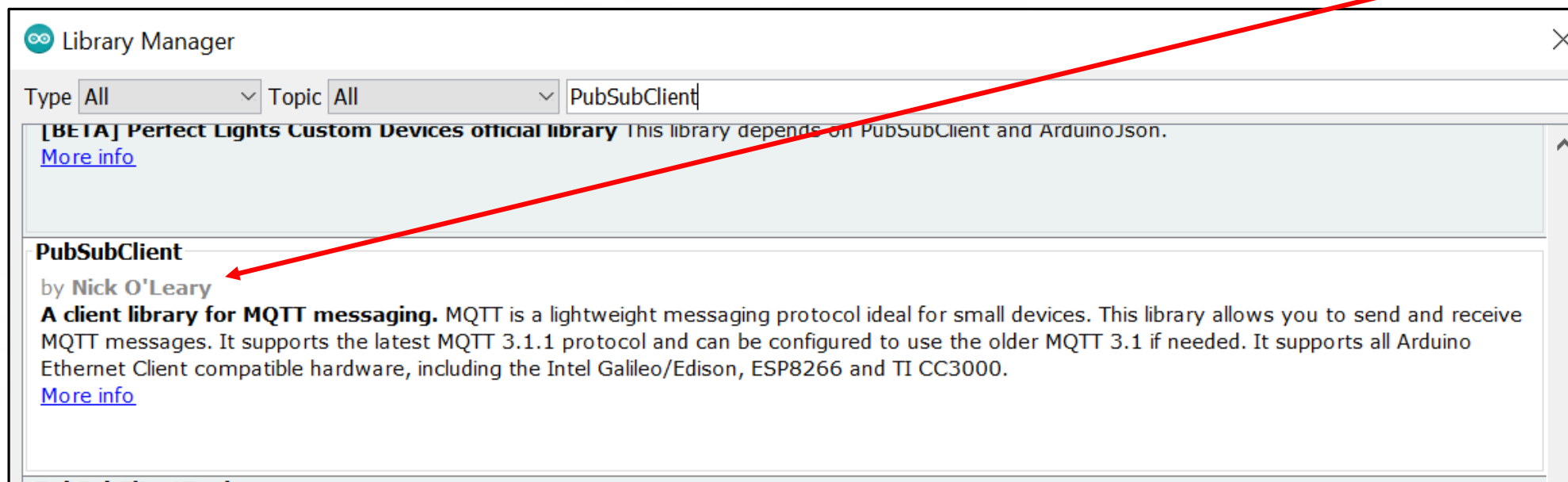
Level		Guarantee	Description
QoS 0	Fire and Forget	At most once	The message is sent once with no confirmation . Fast but unreliable.
QoS 1	Acknowledged Delivery	At least once	The message is guaranteed to arrive , but duplicates may occur if acknowledgments are lost.
QoS 2	Assured Delivery	Exactly once	The message is guaranteed to arrive exactly one time through a multi-step handshake . Slowest but most reliable.

2. Installing a Necessary Library in Arduino IDE

Installing a Necessary Library in Arduino IDE

To let an ESP32 to sent a message by using the MQTT protocol, we must install a library “**PubSubClient.h**” in the Arduino IDE.

- 1) On the Arduino IDE, choose
Sketch → Include Library → Manage Libraries...
- 2) Search for **PubSubClient** and install it. (This library is provided by **Nick O’Leary**.)



- 3) **After finishing, you close the Arduino IDE and open it again.**

3. Publishing/Subscribing via a Public MQTT Broker using Two ESP32

3.1 Project Outline

In this project, we will use two ESP32.

- a) We connect an ESP32 (called “**PubNode**”) to a potentiometer and set it as a **publisher** to send the potentiometer value to the other ESP32 (subscriber) via a public MQTT broker.
- b) We connect an ESP32 (called “**SubNode**”) to an LED and a resistor and set it as a **subscriber** to receive the potentiometer value via a public MQTT broker.

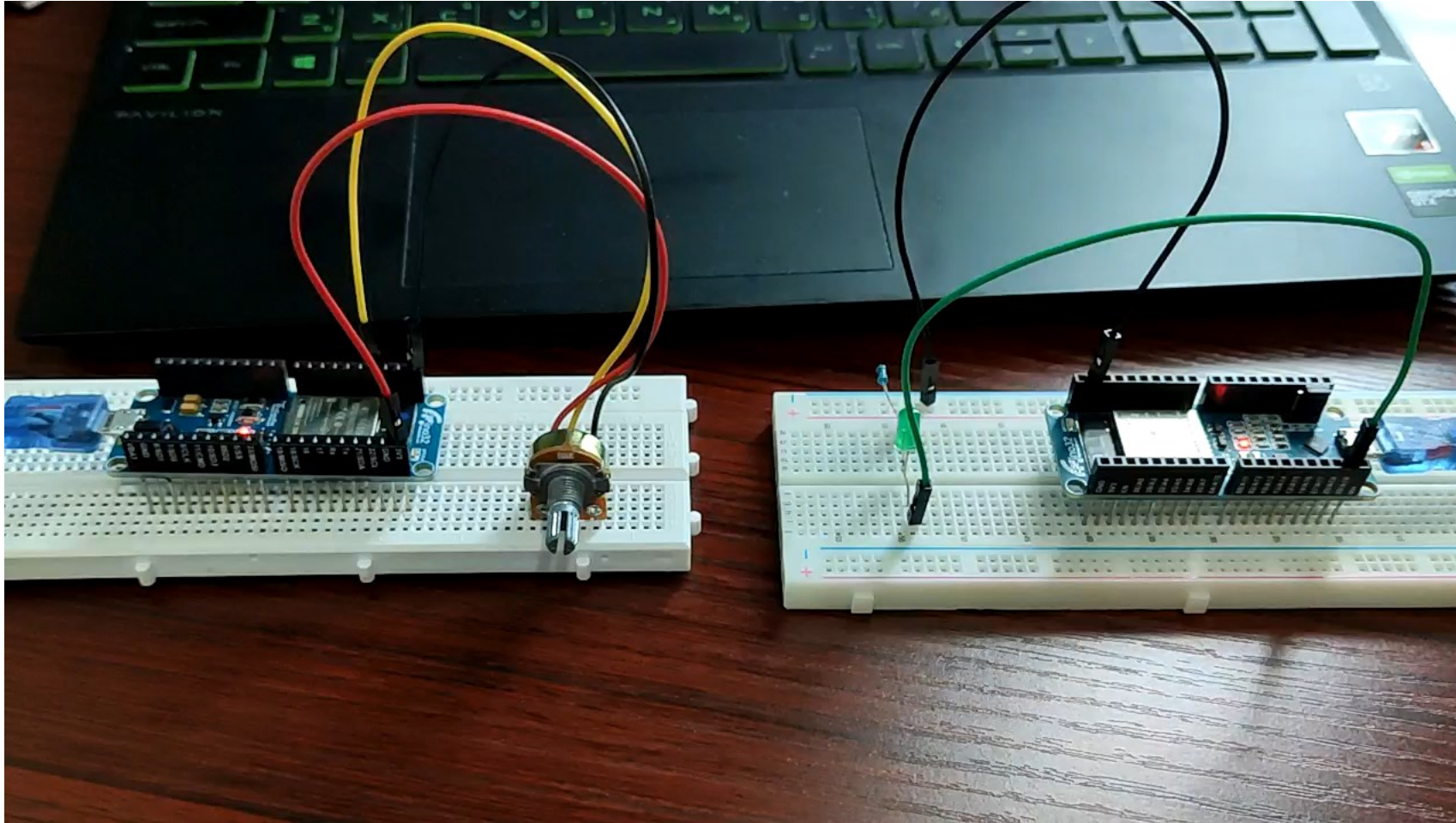
Rotating the potentiometer on the PubNode will turn on/off the LED on the SubNode:

- If the potentiometer is more than 500, the LED will be on; otherwise, the LED will be off.

These two ESP32 do not need to connect to the same access point and they can be at different locations.

3.1 Project Outline

A demonstration video is shown below.



3.1 Project Outline

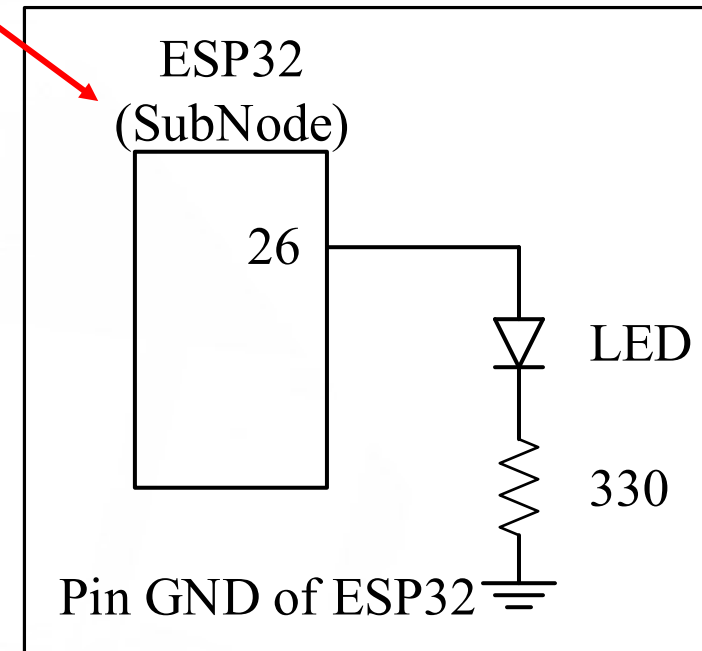
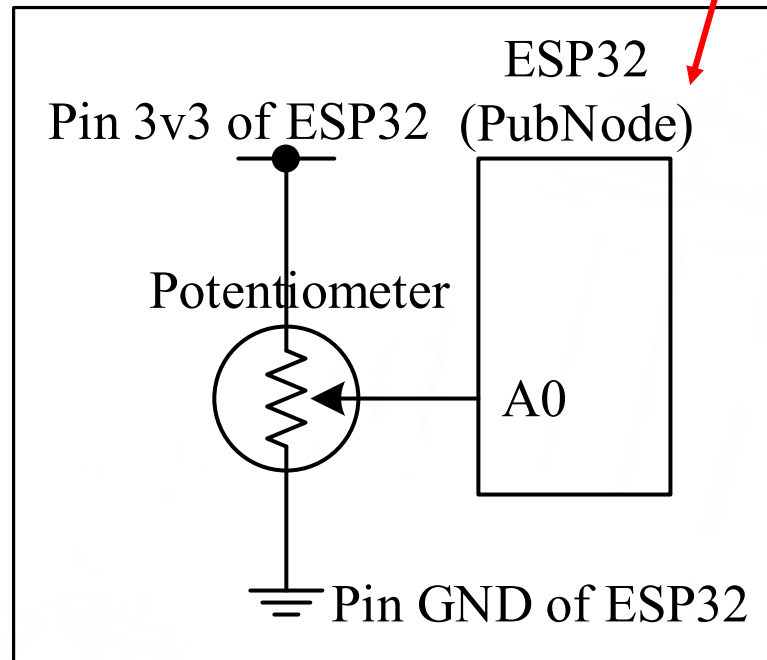
Here is a list of public MQTT brokers which are free (<https://mntolia.com/10-free-public-private-mqtt-brokers-for-testing-prototyping/>), where the broker address and port are specified. In this project, we will use the **HiveMQ** as our public MQTT broker (port **1883**).

- Note that if there is a problem with HiveMQ, you might choose **Mosquitto** instead.

Name	Broker Address	TCP Port
+ Eclipse	mqtt.eclipse.org	1883
+ Mosquitto	test.mosquitto.org	1883
+ HiveMQ	broker.hivemq.com	1883
+ Flespi	mqtt.flespi.io	1883
+ Dioty	mqtt.dioty.co	1883
+ Fluux	mqtt.fluux.io	1883
+ EMQX	broker.emqx.io	1883

3.2 Schematic

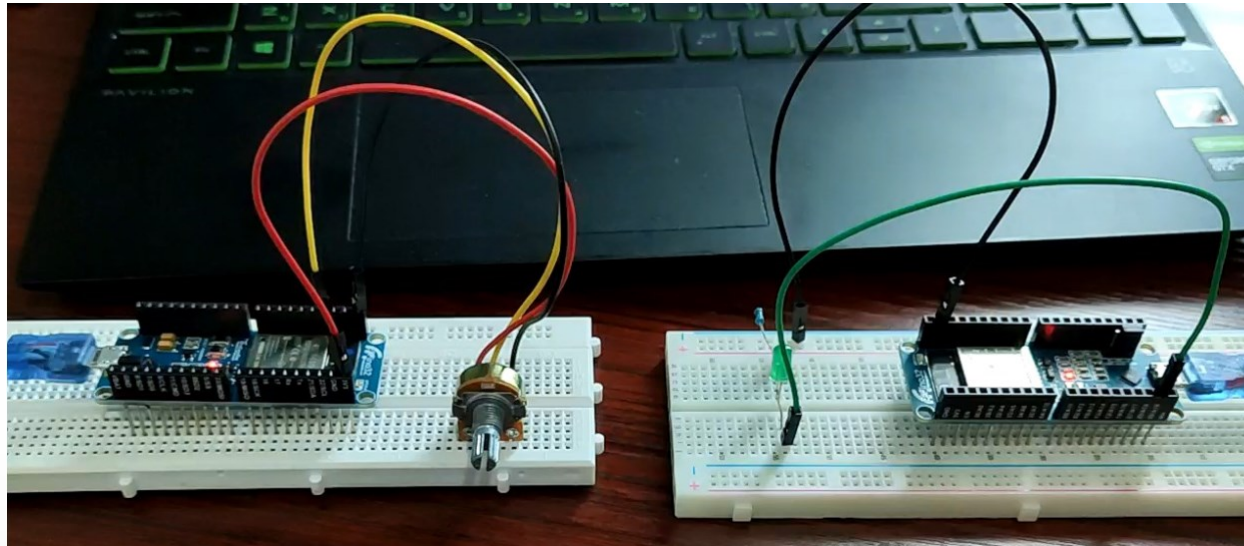
- We connect a potentiometer to the PubNode as shown below.
- We connect an LED and a resistor to the SubNode as shown below.



3.3 Arduino Code

In this project, we have two Arduino codes.

- 1) The Arduino code for the PubNode: “**Public_HiveMQ_PubNode.ino**”. Using this code will set the ESP32 to be a publisher, read the potentiometer value, and send it to the public HiveMQ broker.
- 2) The Arduino code for the SubNode: “**Public_HiveMQ_SubNode.ino**”. Using this code will set the ESP32 to be a subscriber, receive the subscribed message from the public HiveMQ broker, and turn on/off the LED.



3.3 Arduino Code

In this code, we will demonstrate how to:

1) Configure the ESP32 as an MQTT Client

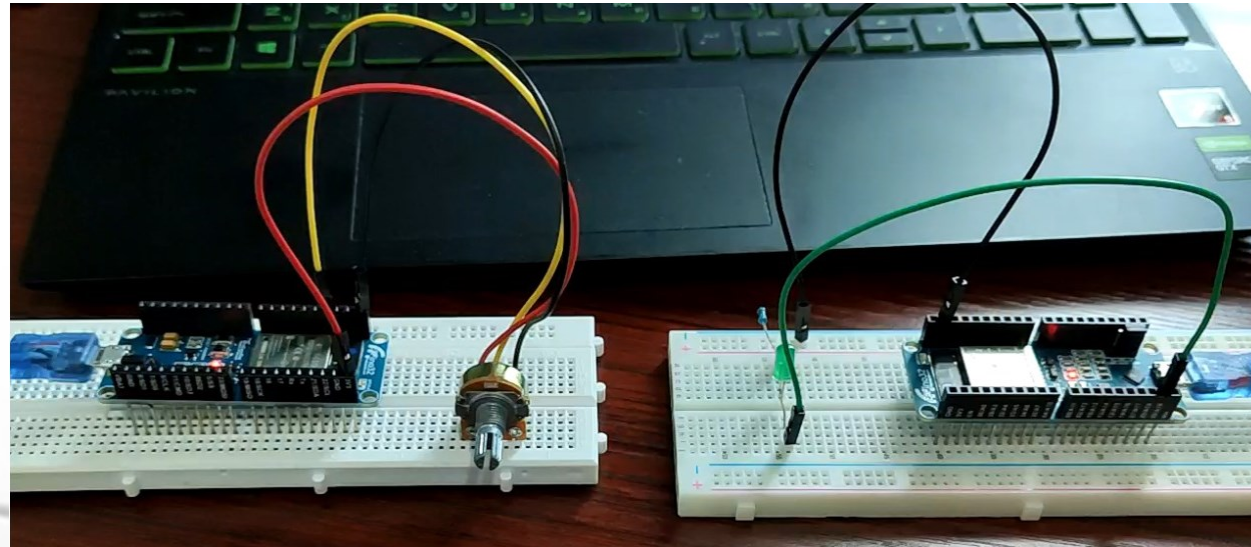
- Set up the connection and define its role as either a Publisher or a Subscriber.

2) Publish Sensor Data

- Read a value from a potentiometer and send it to a specific MQTT topic.

3) Subscribe and Act on Data

- Listen for messages on a topic and use the received data to control a physical output (an LED).



3.3 Arduino Code

Please revise the given two codes as follows.

- 1) Write this code in your Arduino IDE.
- 2) You have to fill in your **SSID** and **Password** of your access point.

```
4 const char* ssid = "Your SSID";  
5 const char* password = "Your Password";
```

- 3) Add your student ID at the end of **Client** and **Topic** such that we have **unique names**.

```
8 const char* ClientID = "Client_YourStudentID";  
9 const char* Topic = "IoT/PotenValue_YourStudentID";
```

- 3.1) *ClentID: PubNode and SubNode have **different** ClientIDs and different from other students.
- 3.2) *Topic: The topic must be the **same topic name** in both PubNode and SubNode. However, it must be a **different name** from other students.

3.3.1 Arduino Code: “Public_HiveMQ_PubNode.ino”

```
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3
4 const char* ssid = "Your SSID";
5 const char* password = "Your Password";
6 const char* mqtt_server = "broker.hivemq.com";
7 const int mqtt_port = 1883;
8 const char* ClientID = "Client_YourStudentID";
9 const char* Topic = "IoT/PotenValue_YourStudentID";
10 long lastMsg = 0;
11
12 WiFiClient ClientNode;
13 PubSubClient ClientESP(ClientNode);
14
```

Necessary libraries

WiFi access point parameters

The MQTT parameters

Set up an MQTT node

3.3.1 Arduino Code: “Public_HiveMQ_PubNode.ino”

```
15 void setup() {  
16     Serial.begin(115200);  
17     Serial.println();  
18     Serial.print("Connecting to ");  
19     Serial.println(ssid);  
20     WiFi.mode(WIFI_STA);  
21     WiFi.begin(ssid, password);  
22     Serial.print("Connecting");  
23     while (WiFi.status() != WL_CONNECTED) {  
24         delay(500);  
25         Serial.print(".");  
26     }  
27     Serial.println();  
28     Serial.print("Connected, IP address: ");  
29     Serial.println(WiFi.localIP());  
30  
31     ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server  
32     ConnectMQTT(); //Connect to the MQTT server.  
33 }  
34
```

→ To connect to your WiFi access point.

→ Specify the MQTT broker and connect to it.

3.3.1 Arduino Code: “Public_HiveMQ_PubNode.ino”

```
35 void loop() {  
36   if (!ClientESP.connected()) {  
37     ConnectMQTT();  
38   }  
39  
40   long now = millis();  
41   if (now - lastMsg > 2000) {  
42     lastMsg = now;  
43     char Val[8];  
44     dtostrf(analogRead(A0), 1, 0, Val);  
45     ClientESP.publish(Topic, Val);  
46     Serial.print("Pub: ");  
47     Serial.println(Val);  
48   }  
49 }  
50
```

To check the connection to the MQTT broker.

This part is as a publisher.

- Check every 2 seconds.
- Publish the value from the pin A0 to the MQTT broker under the topic “Topic”.
- The *dtostrf()* is a function to convert an integer to a string.
- The *.publish()* is to publish.

3.3.1 Arduino Code: “Public_HiveMQ_PubNode.ino”

```
51 void ConnectMQTT() {  
52     while (!ClientESP.connected()) {  
53         Serial.print("Attempting MQTT connection...");  
54         // Attempt to connect to the broker  
55         if (ClientESP.connect(ClientID, NULL, NULL)) {  
56             Serial.println("connected");  
57         } else {  
58             Serial.print("failed, rc=");  
59             Serial.print(ClientESP.state());  
60             Serial.println(" try again in 5 seconds");  
61             // Wait 5 seconds before retrying  
62             delay(5000);  
63         }  
64     }  
65 }
```

A custom function to connect to the MQTT broker and listen to an incoming message from the broker.

Compare to the Arduino code on Page 23, there is no the *.setCallback()* method and the *.subscribe()* method because this ESP32 is working as a publisher only.

3.3.2 Arduino Code: “Public_HiveMQ_SubNode.ino”

```
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3
4 const char* ssid = "Your SSID";
5 const char* password = "Your Password";
6 const char* mqtt_server = "broker.hivemq.com";
7 const int mqtt_port = 1883;
8 const char* ClientID = "Client_YourStudentID";
9 const char* Topic = "IoT/PotenValue_YourStudentID";
10 long lastMsg = 0;
11
12 WiFiClient ClientNode;
13 PubSubClient ClientESP(ClientNode);
14
```

Necessary libraries

WiFi access point parameters

The MQTT parameters

Set up an MQTT node

3.3.2 Arduino Code: “Public_HiveMQ_SubNode.ino”

```
15 void setup() {  
16   Serial.begin(115200);  
17   Serial.println();  
18   Serial.print("Connecting to ");  
19   Serial.println(ssid);  
20   WiFi.mode(WIFI_STA);  
21   WiFi.begin(ssid, password);  
22   Serial.print("Connecting");  
23   while (WiFi.status() != WL_CONNECTED) {  
24     delay(500);  
25     Serial.print(".");  
26   }  
27   Serial.println();  
28   Serial.print("Connected, IP address: ");  
29   Serial.println(WiFi.localIP());  
30  
31   ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server  
32   ClientESP.setCallback(callback);  
33   ConnectMQTT(); //Connect to the MQTT server.  
34  
35   pinMode(26, OUTPUT); //Set the pin mode to the pin GPIO26  
36   digitalWrite(26, LOW);  
37 }  
38
```

To connect to your WiFi access point.

Specify the MQTT broker, set up the call back (for a subscriber), and connect to it.

3.3.2 Arduino Code: “Public_HiveMQ_SubNode.ino”

```
39 void loop() {  
40   if (!ClientESP.connected()) {  
41     ConnectMQTT();  
42   }  
43   ClientESP.loop();  
44 }  
45  
46 void ConnectMQTT() {  
47   while (!ClientESP.connected()) {  
48     Serial.print("Attempting MQTT connection...");  
49     // Attempt to connect to the broker  
50     if (ClientESP.connect(ClientID, NULL, NULL)) {  
51       Serial.println("connected");  
52       ClientESP.setCallback(callback);  
53       ClientESP.subscribe(Topic);  
54     } else {  
55       Serial.print("failed, rc=");  
56       Serial.print(ClientESP.state());  
57       Serial.println(" try again in 5 seconds");  
58       // Wait 5 seconds before retrying  
59       delay(5000);  
60     }  
61   }  
62 }  
63
```

To check the connection to the MQTT broker.

This is for a subscriber. This command should be called regularly to allow the client to process incoming messages and maintain its connection to the server.

A custom function:

- to connect to the MQTT broker
- (as a subscriber) listen to an incoming message from the broker.

3.3.2 Arduino Code: “Public_HiveMQ_SubNode.ino”

```
64 void callback(char* topic, byte* payload, unsigned int length) {  
65     Serial.print("Message arrived [");  
66     Serial.print(topic);  
67     Serial.print("] ");  
68     String ValString;  
69     for (int i = 0; i < length; i++) {  
70         Serial.print((char)payload[i]);  
71         ValString += (char)payload[i];  
72     }  
73     Serial.println();  
74  
75     int ValInt = ValString.toInt();  
76     Serial.print("Sub : ");  
77     Serial.println(ValInt);  
78     Serial.println("===== Wait 2 seconds =====");  
79     if (ValInt > 500) {  
80         digitalWrite(26, HIGH); // Turn the LED on  
81     } else {  
82         digitalWrite(26, LOW); // Turn the LED off  
83     }  
84 }
```

This function will be called if the ESP32 receives an incoming message from the broker.

The incoming message is stored in the variable “ValString”.

- Convert the ValString to an integer.
- Store it in the variable “ValInt”.
- Use the ValInt to turn on/off the LED.

3.4 Code Explanation

The important parts of the code can be explained as follows.

- 1) Declare MQTT parameters: MQTT broker address, MQTT port number (in general, it is 1883), name of the MQTT publisher or subscriber, and the topic.
 - The publisher name, subscriber name, or Topic can be any name.

- Pub Node:

```
6 const char* mqtt_server = "broker.hivemq.com";  
7 const int mqtt_port = 1883;  
8 const char* ClientID = "Client_YourStudentID";  
9 const char* Topic = "IoT/PotenValue_YourStudentID";
```

- Sub Node:

```
6 const char* mqtt_server = "broker.hivemq.com";  
7 const int mqtt_port = 1883;  
8 const char* ClientID = "Client_YourStudentID";  
9 const char* Topic = "IoT/PotenValue_YourStudentID";
```

- 2) Set the ESP32 as a web client and pub/sub client.

```
12 WiFiClient ClientNode;  
13 PubSubClient ClientESP(ClientNode);
```

3.4 Code Explanation

3) Set up the MQTT protocol.

- Specify the MQTT server.
- Specify that if a subscribed message arrives, call the function “callback”.
- Connect to the MQTT broker using a custom function “ConnectMQTT()”.

• Pub Node:

```
31 ClientESP.setServer(mqtt_server, mqtt_port);  
32 ConnectMQTT(); //Connect to the MQTT server.
```

• Sub Node:

```
31 ClientESP.setServer(mqtt_server, mqtt_port);  
32 ClientESP.setCallback(callback);  
33 ConnectMQTT(); //Connect to the MQTT server.
```

4) Check whether the ESP32 connects to the MQTT broker.

Pub Node:

```
36 if (!ClientESP.connected()) {  
37 |   ConnectMQTT();  
38 }
```

Sub Node:

```
40 if (!ClientESP.connected()) {  
41 |   ConnectMQTT();  
42 }
```

3.4 Code Explanation

- 5) In the Sub Node, call this function regularly to allow the client to process incoming messages and maintain its connection to the server.

```
43 ClientESP.loop();
```

- 6) In the Pub Node, convert an integer to a string and publish (send) it under the topic “Topic” to the MQTT broker.

```
44 dtostrf(analogRead(A0), 1, 0, Val);  
45 ClientESP.publish(Topic, Val);
```

3.4 Code Explanation

- 7) In the Sub Node, the topic to be subscribed from the MQTT broker.

```
53 ClientESP.subscribe(Topic);
```

- 8) In the Sub Node, the function “callback” will be called when a subscribed message is received. The subscribed message is stored in the pointer variable “payload” whose length is equal to the variable “length”.

```
64 void callback(char* topic, byte* payload, unsigned int length) {
```

- 9) In the Sub Node, retrieve the subscribed message from the pointer variable “payload” to the variable “ValString”.

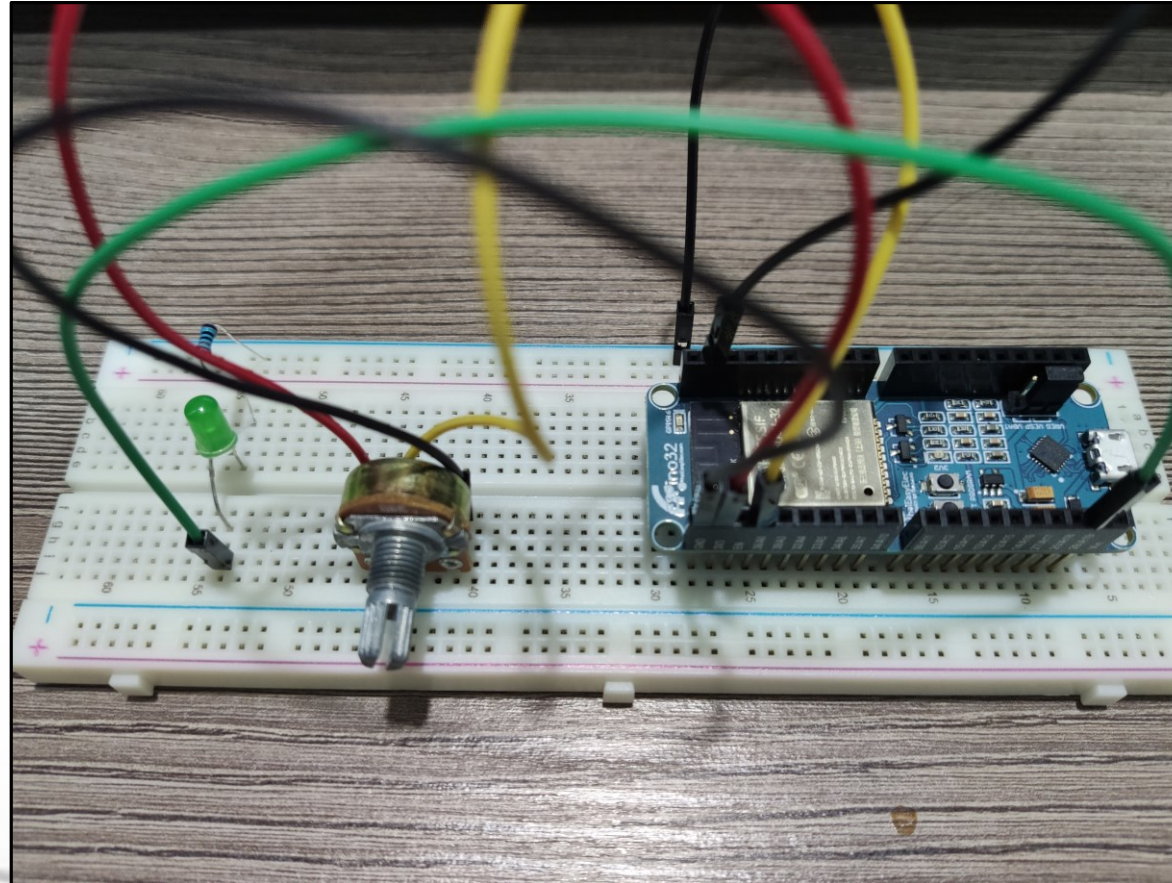
```
68 String ValString;  
69 for (int i = 0; i < length; i++) {  
70     Serial.print((char)payload[i]);  
71     ValString += (char)payload[i];  
72 }
```

4. Publishing/Subscribing via a Public MQTT Broker using One ESP32

4.1 Project Outline

In this project, we will implement a global MQTT-based system, where

- a **public MQTT broker** will be set up as our MQTT broker,
- our ESP32 will work as **both publisher and subscriber** (to send the data to itself).



4.1 Project Outline

A video demonstration is shown below.

```
Public_HiveMQ_PubSub
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3
4 const char* ssid = "MyWiFi";
5 const char* password = "Moon15Star18";
6 const char* mqtt_server = "broker.hivemq.com";
7 const int mqtt_port = 1883;
8 const char* ClientID = "ClientPubSub";
9 const char* Topic = "/IoT/PotenValue";
10 long lastMsg = 0;
11
12 WiFiClient ClientNode;
13 PubSubClient ClientESP(ClientNode);
14
15 void setup() {
```

4.1 Project Outline

Here is a list of public MQTT brokers which are free (<https://mntolia.com/10-free-public-private-mqtt-brokers-for-testing-prototyping/>), where the broker address and port are specified. In this project, we will use the **HiveMQ** as our public MQTT broker (port **1883**).

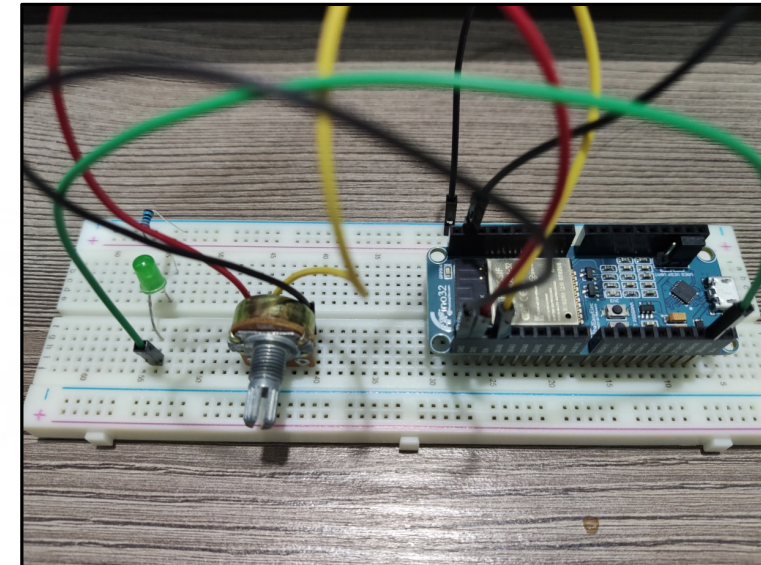
- Note that if there is a problem with HiveMQ, you might choose **Mosquitto** instead.

Name	Broker Address	TCP Port
+ Eclipse	mqtt.eclipse.org	1883
+ Mosquitto	test.mosquitto.org	1883
+ HiveMQ	broker.hivemq.com	1883
+ Flespi	mqtt.flespi.io	1883
+ Dioty	mqtt.dioty.co	1883
+ Fluux	mqtt.fluux.io	1883
+ EMQX	broker.emqx.io	1883

4.1 Project Outline

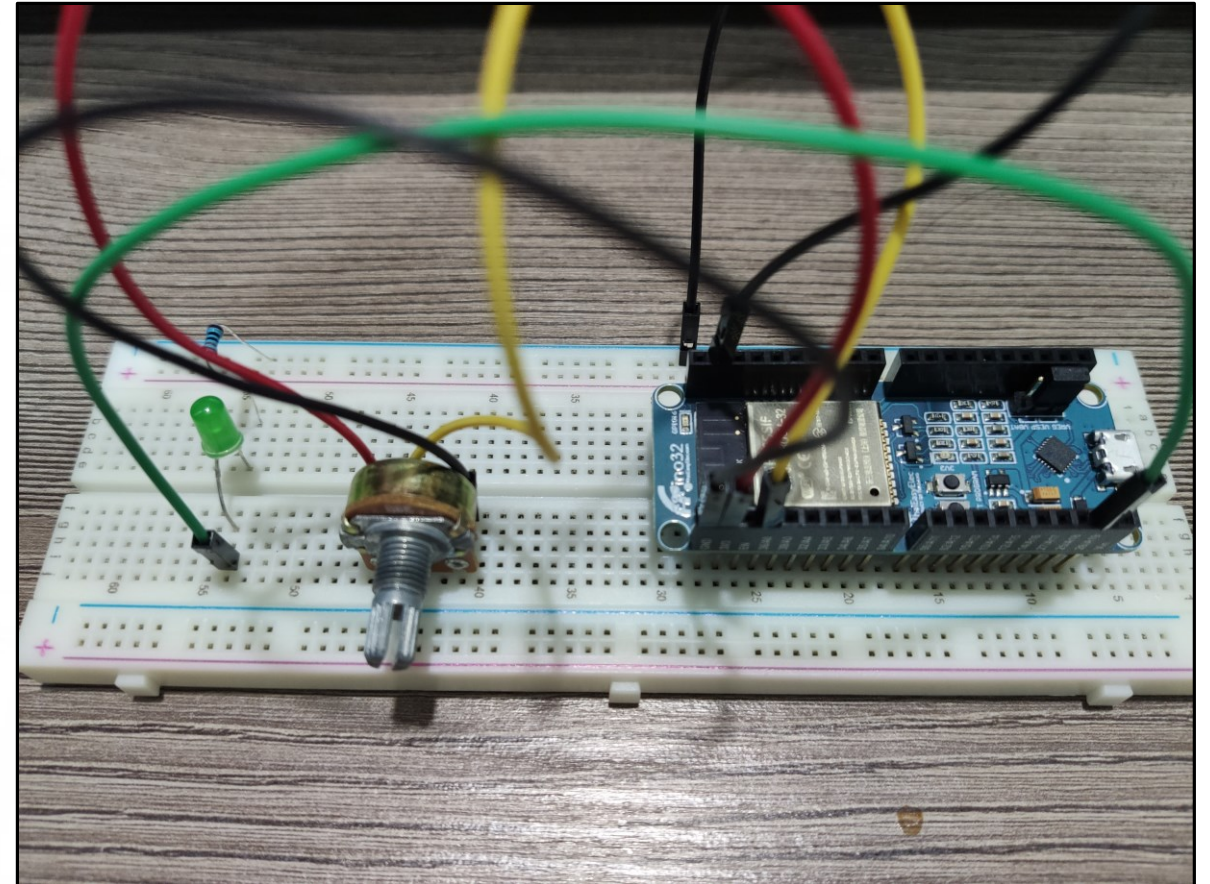
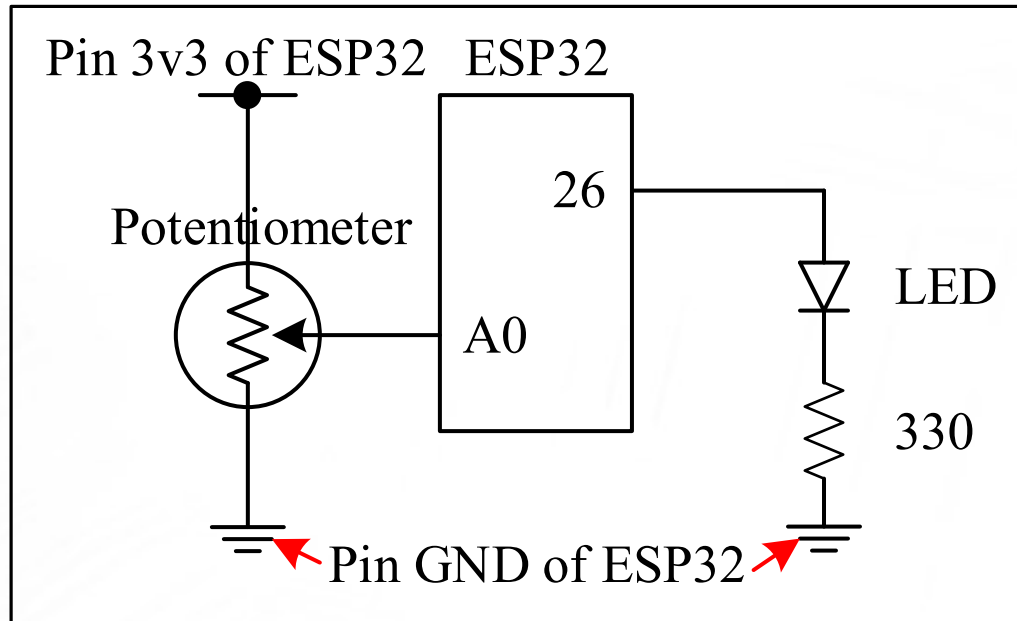
In this project, we will have the following tasks.

- 1) We will connect a **potentiometer** and an **LED (and a resistor)** on the ESP32.
- 2) Rotating the potentiometer will turn on and off the LED.
 - Potentiometer value $> 500 \rightarrow$ LED is on
 - Otherwise $\rightarrow$ LED is off.
- 3) Roles of ESP32:
 - As a **publisher**, ESP32 will send the potentiometer value to the MQTT broker every 2 seconds.
 - As a **subscriber**, ESP32 will receive a message (potentiometer value) from the MQTT broker (when the broker receives a new value). ESP32 compares the received value (> 500 or not) and turns the LED on or off, accordingly.



4.2 Schematic

We connect a potentiometer and an LED to the ESP32 as shown in the following schematic diagram.



4.3 Arduino Code

The Arduino code “**Public_HiveMQ_PubSub.ino**” will set the ESP32 to be both **publisher** (sends the potentiometer value to the broker) and **subscriber** (receives the potentiometer value from the broker) and connect to the HiveMQ broker. Please revise the given code as follows.

- 1) Write this code in your Arduino IDE.
- 2) You have to fill in your SSID and Password of your access point.

```
4 const char* ssid = "Your SSID";  
5 const char* password = "Your Password";
```

- 3) Add your student ID at the end of Client and Topic such that we have **unique names**.

```
8 const char* ClientID = "Client_YourStudentID";  
9 const char* Topic = "IoT/PotenValue_YourStudentID";
```

4.3 Arduino Code

```
1 #include <WiFi.h>
2 #include <PubSubClient.h>
3
4 const char* ssid = "Your SSID";
5 const char* password = "Your Password";
6 const char* mqtt_server = "broker.hivemq.com";
7 const int mqtt_port = 1883;
8 const char* ClientID = "Client_YourStudentID";
9 const char* Topic = "IoT/PotenValue_YourStudentID";
10 long lastMsg = 0;
11
12 WiFiClient ClientNode;
13 PubSubClient ClientESP(ClientNode);
14
```

Necessary libraries

WiFi access point name and password

MQTT Details

Set up an MQTT node

4.3 Arduino Code

```
15 void setup() {  
16   Serial.begin(115200);  
17   Serial.println();  
18   Serial.print("Connecting to ");  
19   Serial.println(ssid);  
20   WiFi.mode(WIFI_STA);  
21   WiFi.begin(ssid, password);  
22   Serial.print("Connecting");  
23   while (WiFi.status() != WL_CONNECTED) {  
24     delay(500);  
25     Serial.print(".");  
26   }  
27   Serial.println();  
28   Serial.print("Connected, IP address: ");  
29   Serial.println(WiFi.localIP());  
30  
31   ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server.  
32   ClientESP.setCallback(callback);  
33   ConnectMQTT(); //Connect to the MQTT server.  
34  
35   pinMode(26, OUTPUT); //Set the pin mode to the pin GPIO26  
36   digitalWrite(26, LOW);  
37 }  
38
```

→ To connect to your WiFi access point.

- To specify the MQTT broker.
- To set up the callback from the MQTT broker.
 - The **callback** function is called when new messages arrive at the client.
- To connect to the MQTT broker.

4.3 Arduino Code

```
39 void loop() {  
40   if (!ClientESP.connected()) {  
41     ConnectMQTT();  
42   }  
43   ClientESP.loop();  
44  
45   long now = millis();  
46   if (now - lastMsg > 2000) {  
47     lastMsg = now;  
48     char Val[8];  
49     dtostrf(analogRead(A0), 1, 0, Val);  
50     ClientESP.publish(Topic, Val);  
51     Serial.print("Pub: ");  
52     Serial.println(Val);  
53   }  
54 }  
55
```

To check the connection to the MQTT broker.

This is for a subscriber. This command should be called regularly to allow the client to process incoming messages and maintain its connection to the server.

This part is as a publisher.

- Check every 2 seconds.
- Publish the value from the pin A0 to the MQTT broker under the topic "Topic".
- The `dtostrf()` is a function to convert an integer to a string.
- The `.publish()` is to publish.

4.3 Arduino Code

```
56 void ConnectMQTT() {  
57     while (!ClientESP.connected()) {  
58         Serial.print("Attempting MQTT connection...");  
59         // Attempt to connect to the broker  
60         if (ClientESP.connect(ClientID, NULL, NULL)) {  
61             Serial.println("connected");  
62             ClientESP.setCallback(callback);  
63             ClientESP.subscribe(Topic);  
64         } else {  
65             Serial.print("failed, rc=");  
66             Serial.print(ClientESP.state());  
67             Serial.println(" try again in 5 seconds");  
68             // Wait 5 seconds before retrying  
69             delay(5000);  
70         }  
71     }  
72 }  
73
```

A custom function:

- to connect to the MQTT broker,
- listen to an incoming message from the broker.

4.3 Arduino Code

```
74 void callback(char* topic, byte* payload, unsigned int length) {  
75     Serial.print("Message arrived [");  
76     Serial.print(topic);  
77     Serial.print("] ");  
78     String ValString;  
79     for (int i = 0; i < length; i++) {  
80         Serial.print((char)payload[i]);  
81         ValString += (char)payload[i];  
82     }  
83     Serial.println();  
84  
85     int ValInt = ValString.toInt();  
86     Serial.print("Sub : ");  
87     Serial.println(ValInt);  
88     Serial.println("===== Wait 2 seconds =====");  
89     if (ValInt > 500) {  
90         digitalWrite(26, HIGH); // Turn the LED on  
91     } else {  
92         digitalWrite(26, LOW); // Turn the LED off  
93     }  
94 }
```

This function will be called if the ESP32 receives an incoming message from the broker.

The incoming message is stored in the variable “ValString”.

- Convert the ValString to an integer.
- Store it in the variable “ValInt”.
- Use the ValInt to turn on/off the LED.

4.4 Code Explanation

The important parts of the code can be explained as follows.

- 1) Declare MQTT parameters: MQTT broker address, MQTT port number (in general, it is 1883), name of the MQTT publisher or subscriber, and the topic.
 - The publisher name, subscriber name, or Topic can be any name.

```
6 const char* mqtt_server = "broker.hivemq.com";  
7 const int mqtt_port = 1883;  
8 const char* ClientID = "Client_YourStudentID";  
9 const char* Topic = "IoT/PotenValue_YourStudentID";
```

- 2) Set the ESP32 as a web client and pub/sub client.

```
12 WiFiClient ClientNode;  
13 PubSubClient ClientESP(ClientNode);
```

4.4 Code Explanation

3) Set up the MQTT protocol.

- Specify the MQTT server.
- Specify that if a subscribed message arrives, call the function “callback”.
- Connect to the MQTT broker using a custom function “ConnectMQTT();”.

```
31 ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server
32 ClientESP.setCallback(callback);
33 ConnectMQTT(); //Connect to the MQTT server.
```

4) Check whether the ESP32 connects to the MQTT broker.

```
40 if (!ClientESP.connected()) {
41     ConnectMQTT();
42 }
```

5) Call this function regularly to allow the client to process incoming messages and maintain its connection to the server.

```
43 ClientESP.loop();
```


4.4 Code Explanation

- 6) Convert an integer to a string and publish (send) it under the topic “Topic” to the MQTT broker.

```
49    dtostrf(analogRead(A0), 1, 0, Val);  
50    ClientESP.publish(Topic, Val);
```

- 7) Declare the topic to be subscribed from the MQTT broker.

```
63    ClientESP.subscribe(Topic);
```

- 8) The function “callback” will be called when a subscribed message is received. The subscribed message is store in the pointer variable “payload” whose length is equal to the variable “length”.

```
74 void callback(char* topic, byte* payload, unsigned int length) {
```

4.4 Code Explanation

- 9) Retrieve the subscribed message from the pointer variable “payload” to the variable “ValString”.

```
79  for (int i = 0; i < length; i++) {  
80      Serial.print((char)payload[i]);  
81      ValString += (char)payload[i];  
82  }
```

Questions?